

Examen Teórico

Programación para Sistemas

Escuela Técnica Superior de Ingenieros Informáticos
Universidad Politécnica de Madrid

Curso 2022/2023 - Julio 2023

Normas

- Se deberá rellenar **apellidos, nombre y número de matrícula** en cada hoja.
- Se deberá tener el **DNI** o el carnet de la UPM en **lugar visible**.
- El examen consta de **13 preguntas** que suman un total de **21 puntos**.
- La duración total del mismo es de **30 minutos**.
- Las preguntas de **tipo test** sólo tiene **una opción válida**. Si se marca más de una opción considerará una respuesta incorrecta. Toda **respuesta incorrecta restará** la puntuación de la pregunta dividida por el número de opciones. Toda pregunta no contestada no restará. Las preguntas de respuesta libre no restarán.
- Las calificaciones se darán a conocer a través del Moodle de la asignatura y la solución al examen se proporcionará antes de la revisión.

Cuestionario

(1½ puntos) 1. El manual de la función `calloc` en la biblioteca `<stdlib.h>` dice:

```
SINOPSIS
#include <stdlib.h>
void *calloc(size_t nmemb, size_t size);
[...]
calloc() asigna memoria para una matriz de nmemb elementos de
size bytes cada uno y devuelve un puntero a la memoria asignada.
La memoria es puesta a cero.
[...]
```

Dado el siguiente programa (asúmanse los *includes* adecuados):

```
int main(void)
{
    char *s = (char*)calloc(sizeof(char), 10);
    printf("%lu\n", strlen(s));
    free(s);
    return 1;
}
```

Se pide señalar la respuesta correcta suponiendo que la llamada a `calloc` no falla.

- A. La salida estándar del programa es 0.
- B. La salida estándar del programa es 10.
- C. No es posible conocer la salida estándar al no haber inicializado el vector `s`.

- (1½ puntos) 2. A continuación se presenta una porción de código C que define el tipo `pila_t` para implementar pilas con cadenas enlazadas, y una función `push` para apilar un nuevo elemento en una pila (asúmanse los *includes* adecuados y que el argumento formal `pp` $\neq$ NULL).

```
typedef struct node_s {
    int top;
    struct node_s *rest;
} *pila_t;

void push(pila_t *pp, int x) {
    pila_t nueva = (pila_t)malloc(sizeof(struct node_s));
    nueva->top = x;
    nueva->rest = *pp;
    *pp = nueva;
}
```

Se pide señalar la respuesta correcta suponiendo que la llamada a `malloc` no falla.

- A. El código no presenta ningún problema.
- B. El código no compila.
- C. Al ejecutar, la variable `nueva` quedará apuntando a un nodo que ya no se podrá liberar (*memory leak*).

- (1 punto) 3. Asumiendo el código de la pregunta 2.

Se pide señalar la respuesta correcta:

- A. La llamada `push(NULL, 42)` devolverá una pila con un único elemento 42.
- B. La llamada `push(NULL, 42)` provocará un error en tiempo de ejecución.

- (2 puntos) 4. Teniendo en cuenta la implementación de la función `sumar` (el tipo `size_t` es un entero sin signo que se usa en C para hablar de tamaños de memoria y longitud de vectores):

```
void sumar(int u[], size_t nu, int v[], size_t nv, int w[]) {
    int *x;
    size_t i, n;
    if (nu < nv) n = nu; else n = nv;
    for (i = 0; i < n; ++i) w[i] = u[i] + v[i];
    if (nu > nv) n = nu; else n = nv;
    if (nu > nv) x = u; else x = v;
    while (i < n) {
        w[i] = x[i];
        i++;
    }
}
```

y suponiendo el siguiente código:

```
int u[] = { 5, 4, 3, 2, 1, 0, -1};
int v[] = { 1, 2, 3, 4, 5 };
int w[7];
sumar(u, 7, v, 5, w);
```

Se pide indicar los valores almacenados en el vector `w` después de su ejecución.

- A. {6, 6, 6, 6, 6, 0, -1}.
- B. {6, 6, 6, 6, 6}.
- C. {2, 4, 6, 8, 10, -1, 0}.
- D. La función no compila.

- (1½ puntos) 5. El manual de la función `scanf` en la biblioteca `<stdlib.h>` dice:

SINOPSIS

```
#include <stdio.h>
int scanf(const char *format, ...);
[...]
```

La familia `scanf` de funciones escudriña la entrada según un formato como se describe más adelante. Este formato puede contener especificadores de conversión; los resultados de tales conversiones, si las hay, se guardan donde apunten los argumentos punteros. La función `scanf` lee la entrada del flujo de entrada estándar `stdin`.

[...]

Están disponibles las siguientes conversiones:

f . Concuerda con un número en coma flotante con signo opcional; el siguiente puntero debe serlo a `float`.

Supóngase la existencia de una variable **float** `x`.

Se pide señalar la invocación correcta de `scanf` para leer un número en coma flotante de la entrada estándar.

A. `scanf("%f", x)`. **B. `scanf("%f", &x)`**. C. `scanf("%f", *x)`.

- (1½ puntos) 6. Dado el siguiente código (el especificador `c` imprime un carácter y el especificador `s` imprime un *string*):

```
char s[] = "Hola mundo";
printf("%c_%s", s[5], s+5);
```

Se pide indicar cuál será el resultado en la salida estándar del `printf`.

A. `m_mundo` B. `mundo_mundo` C. `Hola_mundo`

- (1½ puntos) 7. Supóngase una variable entera **int** `next` cuyo valor está entre 0 y 7. Dado el siguiente código:

```
next = (next + 1) % 8;
```

Se pide indicar qué sentencia tiene un **significado distinto** al código anterior:

- A. **`if (next == 7) next = 0; else next = next + 1;`**
B. **`if (next == 8) next = 0; else next = next + 1;`**
C. **`if (next == 7) next = 0; else ++next;`**

- (2 puntos) 8. Teniendo en cuenta la implementación de la función `intercambiar`:

```
void intercambiar(int *x, int *y) {
    int aux = *x;
    *x = *y;
    *y = aux;
}
```

y suponiendo la existencia de un vector de enteros **int** `x[10]`.

Se pide indicar la sentencia que intercambia al primer y el último elemento del vector `x`.

- A. `intercambiar(x[0], x[9])`.
B. `intercambiar(&x, &x+9)`.
C. `intercambiar(x, x+9)`.
D. `intercambiar(&x, &(x+9))`.

- (1½ puntos) 9. C es un lenguaje que no tiene tipo de datos *booleano*. La forma de representar condiciones en **if**, por ejemplo, es utilizando alguno de los tipos *entero*¹.

Se pide señalar la afirmación correcta.

- A. Se considera *false* el valor 0 y *true* cualquier otro valor entero.
- B. Se considera *false* el valor negativo y el 0 y *true* cualquier valor positivo.
- C. Se considera *false* el valor 0 y *true* el valor 1 (cualquier otro valor provoca un error en tiempo de ejecución).

- (1½ puntos) 10. Se pide señalar la respuesta correcta al respecto de lo que hace el parámetro \$# en Bash:

- A. Expande al número de parámetros del *script*.
- B. Expande a todos los parámetros del *script* separados por espacios.
- C. Expande al nombre del *script*.

- (1½ puntos) 11. ¿Cuál de los siguientes mandatos comprueba si el fichero mifichero existe y tiene permiso de escritura?

- A. **test** -e mifichero
- B. **test** -w mifichero
- C. **test** -x mifichero

- (2 puntos) 12. Dado el siguiente script de Bash 123.sh (el mandato **echo** con opción -n no imprime un cambio de línea):

```
#!/bin/bash
function f() {
    echo -n 1
    echo -n 2
}
echo -n 3
f
```

¿Cuál es el resultado de la siguiente invocación desde la línea de mandatos?

\$./123.sh

Se pide señalar la respuesta correcta:

- A. 123 B. 12312 C. 321 D. 312

- (2 puntos) 13. El programa **tail** usado sin argumentos *muestra por la salida estándar las últimas 10 líneas de la entrada estándar*, el programa **cat** con un argumento interpreta dicho argumento como el nombre de un fichero y saca su contenido por la salida estándar, el fichero /var/log/syslog es un fichero que tiene registrado todo suceso del sistema operativo.

Se pide usando **tail** y **cat** bajo las premisas anteriores así como la redirección de la salida estándar, escribir en una sola línea un mandato Bash que guarde en un fichero final.txt 10 últimas líneas del fichero /var/log/syslog (préstese atención a la hora de escribir correctamente los nombres de los ficheros y de los mandatos).

Solución: **cat** /var/log/syslog | **tail** > final.txt

¹ Escalar para ser más preciso, aunque esto no importa para señalar la respuesta correcta.